E-commerce Database Analytics

Capstone Project Report: schema design, SQL analytics, indexing, views and window functions (MySQL 8.0)

Student name	[Your Name]
Roll number	[Your Roll Number]
Assigned problem statement no.	[Problem Statement No.]
GitHub repository	https://github.com/your-github-username/ecommerce-sql-analytics
Database / tools	MySQL 8.0 (InnoDB), SQL only; ER diagram with Graphviz
Report date	October 30, 2026

About this report. Every SQL statement in this document was executed on a real MySQL 8.0 server and the outputs shown are the actual results (long result sets are shortened here; complete outputs are in the `outputs/` folder of the repository). The data is synthetic and generated by SQL so the project is fully reproducible.

Requirements covered

Requirement	Where	Evidence in this report
Schema for product listings, seller information and customer reviews	Section 1	7 tables with PK/FK, CHECK and UNIQUE constraints; ER diagram
Queries: sales trends, top-selling products, customer satisfaction scores	Section 3	Monthly trend, top-10 products, per-product and per-category satisfaction (incl. Bayesian score)
Cohort analysis for customer behaviour and marketing	Section 4	Retention matrix, pivot, cumulative revenue per cohort, days to second order
Composite indexes on orders(customer_id, order_date), products(category_id, price); FULLTEXT on title+description; EXPLAIN comparisons; low-selectivity example	Section 5	Before/after EXPLAIN and EXPLAIN ANALYZE; status index shown to be slower for a 70% value
Seller performance view, customer lifetime value view, materialized-view simulation, view vs summary table trade-off	Section 6	Two views, summary table + stored procedure + scheduled event, timing comparison, trade-off table
Window functions: RANK, LAG, 3-month moving average, MIN OVER cohort, NTILE(4)	Section 7	All five implemented and executed

Results at a glance

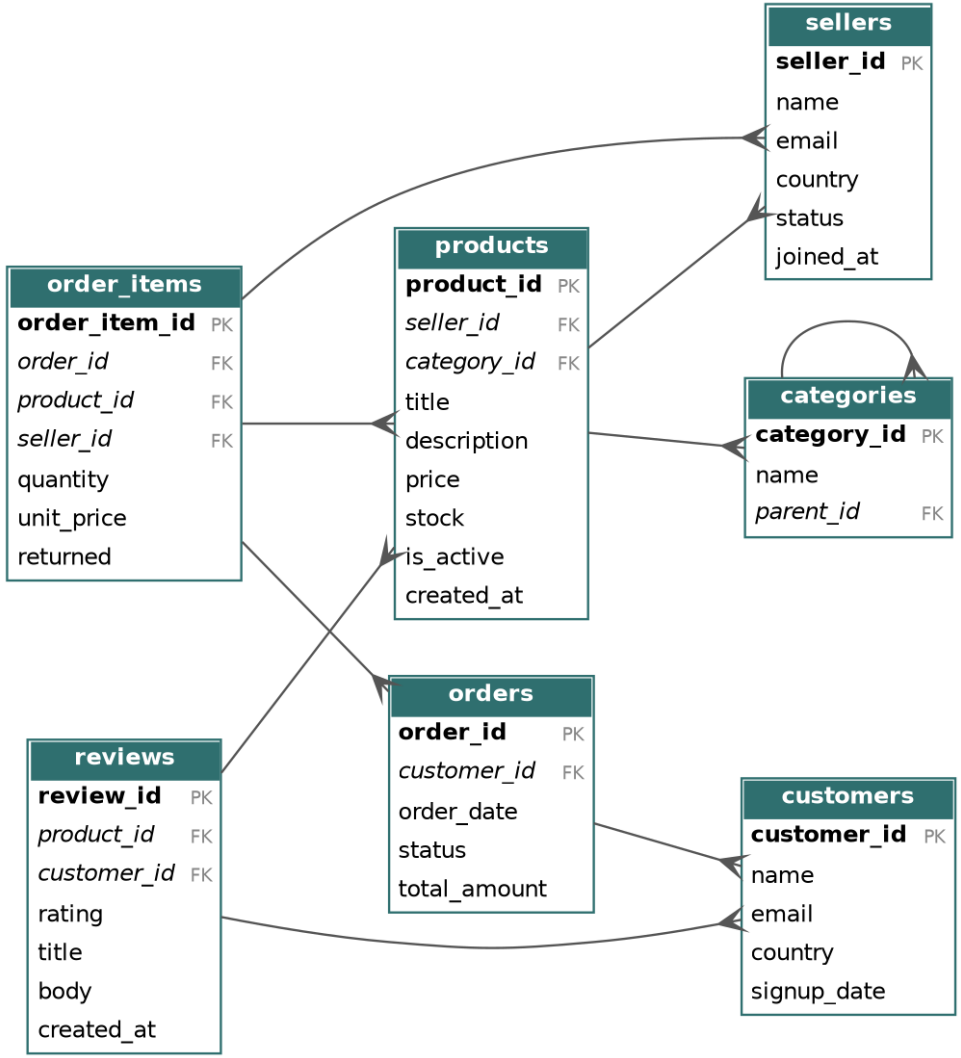
Query	Without optimisation	With optimisation	Speed-up
Orders of one customer in a date range (composite index customer_id, order_date)	8.94 ms	0.20 ms	44x
Products in a category within a price range (composite index category_id, price)	0.92 ms	0.32 ms	2.8x
Keyword search 'wireless' (LIKE '%...%' vs FULLTEXT) - only 2,000 rows, so gap is small	2.54 ms	1.71 ms	1.5x
Top 10 sellers: view v_seller_performance vs summary table	289.1 ms	0.31 ms	943x

Averages of 30 runs on MySQL 8.0 inside the same session (warm cache). Absolute times depend on the machine; the point is the relative difference.

Key findings from the (synthetic) data

- Best-selling product by net revenue: **Organic Keyboard 10** (Sports), 169,572.
- Largest seller: **Seller 1** with 1,969,695 net sales, average rating 3.41, return rate 3.4%.
- Average cohort retention: 79.7% after 1 month, 61.9% after 3 months, 37.6% after 6 months.
- Top 25% of customers generate 41.2% of net revenue.
- An index on a low-selectivity column (order status) made the 'delivered' query about twice as slow as a table scan; composite indexes gave large speed-ups.

Entity-relationship diagram



Bold = primary key, italic = foreign key. order_items.seller_id is a deliberate denormalisation (seller at time of sale).

1. Schema design

Seven tables model the marketplace: **categories** (with an optional parent for a category tree), **sellers**, **customers**, **products**, **orders**, **order_items** and **reviews**. Foreign keys protect integrity, CHECK constraints guard prices, quantities and ratings (1-5), and a UNIQUE key on (product_id, customer_id) allows one review per customer per product.

Design decisions: **order_items.unit_price** stores the price at the time of sale so later price changes do not rewrite history; **order_items.seller_id** is intentionally denormalised so seller sales do not need an extra join; **order_items.returned** records returns per line, which makes seller return rates simple to compute.

SQL

```
DROP DATABASE IF EXISTS shop
```

OUTPUT

```
Query OK, 13 rows affected (0.14 sec)
```

SQL

```
CREATE DATABASE shop CHARACTER SET utf8mb4 COLLATE utf8mb4_0900_ai_ci
```

OUTPUT

```
Query OK, 1 row affected (0.00 sec)
```

SQL

```
CREATE TABLE categories (
  category_id  INT UNSIGNED NOT NULL AUTO_INCREMENT,
  name         VARCHAR(80)  NOT NULL,
  parent_id    INT UNSIGNED NULL,
  PRIMARY KEY (category_id),
  UNIQUE KEY uq_category_name (name),
  CONSTRAINT fk_categories_parent FOREIGN KEY (parent_id) REFERENCES categories (category_id)
) ENGINE=InnoDB
```

OUTPUT

```
Query OK, 0 rows affected (0.01 sec)
```

SQL

```
CREATE TABLE sellers (
  seller_id  INT UNSIGNED NOT NULL AUTO_INCREMENT,
  name       VARCHAR(120) NOT NULL,
  email      VARCHAR(255) NOT NULL,
  country    CHAR(2)      NOT NULL,
  status     ENUM('active','suspended','closed') NOT NULL DEFAULT 'active',
  joined_at  DATE         NOT NULL,
  PRIMARY KEY (seller_id),
  UNIQUE KEY uq_seller_email (email)
) ENGINE=InnoDB
```

OUTPUT

```
Query OK, 0 rows affected (0.01 sec)
```

SQL

```
CREATE TABLE customers (
  customer_id  INT UNSIGNED NOT NULL AUTO_INCREMENT,
  name         VARCHAR(120) NOT NULL,
  email        VARCHAR(255) NOT NULL,
  country      CHAR(2)      NOT NULL,
  signup_date  DATE         NOT NULL,
  PRIMARY KEY (customer_id),
  UNIQUE KEY uq_customer_email (email)
) ENGINE=InnoDB
```

OUTPUT

```
Query OK, 0 rows affected (0.01 sec)
```

SQL

```
CREATE TABLE products (
  product_id   INT UNSIGNED NOT NULL AUTO_INCREMENT,
  seller_id    INT UNSIGNED NOT NULL,
  category_id  INT UNSIGNED NOT NULL,
  title        VARCHAR(200) NOT NULL,
  description  TEXT         NOT NULL,
  price        DECIMAL(10,2) NOT NULL CHECK (price >= 0),
  stock        INT UNSIGNED NOT NULL DEFAULT 0,
  is_active    TINYINT(1)   NOT NULL DEFAULT 1,
  created_at   DATE         NOT NULL,
  PRIMARY KEY (product_id),
  CONSTRAINT fk_products_seller   FOREIGN KEY (seller_id)   REFERENCES sellers (seller_id),
  CONSTRAINT fk_products_category FOREIGN KEY (category_id) REFERENCES categories (category_id)
) ENGINE=InnoDB
```

OUTPUT

```
Query OK, 0 rows affected (0.01 sec)
```

SQL

```
CREATE TABLE orders (
  order_id      BIGINT UNSIGNED NOT NULL AUTO_INCREMENT,
  customer_id   INT UNSIGNED NOT NULL,
  order_date    DATE         NOT NULL,
  status        ENUM('placed','shipped','delivered','returned','cancelled') NOT NULL,
  total_amount  DECIMAL(12,2) NOT NULL DEFAULT 0,
  PRIMARY KEY (order_id),
  CONSTRAINT fk_orders_customer FOREIGN KEY (customer_id) REFERENCES customers (customer_id)
) ENGINE=InnoDB
```

OUTPUT

```
Query OK, 0 rows affected (0.01 sec)
```

SQL

```
CREATE TABLE order_items (
  order_item_id  BIGINT UNSIGNED NOT NULL AUTO_INCREMENT,
  order_id       BIGINT UNSIGNED NOT NULL,
  product_id     INT UNSIGNED NOT NULL,
  seller_id      INT UNSIGNED NOT NULL,          -- denormalised: seller at time of sale
  quantity       INT UNSIGNED NOT NULL CHECK (quantity > 0),
  unit_price     DECIMAL(10,2) NOT NULL,         -- price at time of sale
  returned       TINYINT(1)   NOT NULL DEFAULT 0,
  PRIMARY KEY (order_item_id),
  CONSTRAINT fk_items_order   FOREIGN KEY (order_id)   REFERENCES orders (order_id),
  CONSTRAINT fk_items_product FOREIGN KEY (product_id) REFERENCES products (product_id),
  CONSTRAINT fk_items_seller  FOREIGN KEY (seller_id)  REFERENCES sellers (seller_id)
) ENGINE=InnoDB
```

OUTPUT

```
Query OK, 0 rows affected (0.02 sec)
```

SQL

```
CREATE TABLE reviews (
  review_id    BIGINT UNSIGNED NOT NULL AUTO_INCREMENT,
  product_id   INT UNSIGNED NOT NULL,
  customer_id  INT UNSIGNED NOT NULL,
  rating       TINYINT UNSIGNED NOT NULL CHECK (rating BETWEEN 1 AND 5),
  title        VARCHAR(150) NULL,
  body         TEXT NULL,
  created_at   DATE NOT NULL,
  PRIMARY KEY (review_id),
  UNIQUE KEY uq_review_once (product_id, customer_id),   -- one review per customer per product
  CONSTRAINT fk_reviews_product  FOREIGN KEY (product_id)  REFERENCES products (product_id),
  CONSTRAINT fk_reviews_customer FOREIGN KEY (customer_id) REFERENCES customers (customer_id)
) ENGINE=InnoDB
```

OUTPUT

```
Query OK, 0 rows affected (0.01 sec)
```

2. Sample data

The data is synthetic and generated with SQL only, using a deterministic hash function (`rnd()`) so every run produces identical rows: 50 sellers, 5,000 customers, 2,000 products, 50,000 orders, 100,000 order lines and 9,994 reviews. Popular products, big sellers and heavy buyers are skewed on purpose, order dates follow signup dates with a decaying delay (so retention curves look realistic), and order status is 70% delivered / 15% shipped / 10% returned / 5% cancelled, which gives a deliberately **low-selectivity** column for the indexing demo.

A base view `v_sales_lines` defines **net revenue** once: cancelled orders are excluded and returned lines count as 0. All later analytics reuse it so the numbers are consistent.

helper number sequence 1..100000

SQL

```
CREATE TABLE seq (n INT UNSIGNED PRIMARY KEY)
```

OUTPUT

```
Query OK, 0 rows affected (0.01 sec)
```

SQL

```
SET SESSION cte_max_recursion_depth = 100000
```

OUTPUT

```
Query OK, 0 rows affected (0.00 sec)
```

SQL

```
INSERT INTO seq
WITH RECURSIVE g (n) AS (SELECT 1 UNION ALL SELECT n + 1 FROM g WHERE n < 100000)
SELECT n FROM g
```

OUTPUT

```
Query OK, 100000 rows affected (0.35 sec)
```

SQL

```
INSERT INTO categories (name) VALUES
 ('Electronics'),('Home & Kitchen'),('Sports'),('Books'),
 ('Beauty'),('Toys'),('Fashion'),('Garden')
```

OUTPUT

```
Query OK, 8 rows affected (0.00 sec)
```

SQL

```
INSERT INTO sellers (name, email, country, joined_at)
SELECT CONCAT('Seller ', n), CONCAT('seller', n, '@example.com'),
       ELT(1 + MOD(n, 4), 'IN', 'US', 'GB', 'DE'),
       DATE_ADD('2022-01-01', INTERVAL MOD(n * 11, 300) DAY)
FROM seq WHERE n <= 50
```

OUTPUT

```
Query OK, 50 rows affected (0.00 sec)
```

Deterministic pseudo-random number in [0,1): same inputs always give the same output, so every rerun produces identical data (RAND(seed) is NOT used: sequential seeds correlate).

SQL

```
CREATE FUNCTION rnd(salt VARCHAR(16), n BIGINT) RETURNS DOUBLE DETERMINISTIC NO SQL
  RETURN CONV(SUBSTRING(MD5(CONCAT(salt, ':', n)), 1, 8), 16, 10) / 4294967296
```

OUTPUT

```
Query OK, 0 rows affected (0.00 sec)
```

SQL

```
INSERT INTO customers (name, email, country, signup_date)
SELECT CONCAT('Customer ', n), CONCAT('customer', n, '@example.com'),
       ELT(1 + MOD(n, 4), 'IN', 'US', 'GB', 'DE'),
       DATE_ADD('2023-01-01', INTERVAL FLOOR(rnd('su', n) * 700) DAY)
FROM seq WHERE n <= 5000
```

OUTPUT

```
Query OK, 5000 rows affected (0.10 sec)
```

SQL

```
INSERT INTO products (seller_id, category_id, title, description, price, stock, created_at)
SELECT 1 + FLOOR(50 * POW(rnd('ps', n), 1.4)),            -- some sellers are bigger than others
       1 + MOD(n, 8),
       CONCAT(ELT(1 + MOD(n, 6), 'Wireless','Ergonomic','Stainless','Portable','Organic','Premium'), ' ',
              ELT(1 + MOD(n DIV 6, 6), 'Headphones','Keyboard','Bottle','Speaker','Lamp','Backpack'),
              ' ', n),
       CONCAT('A ', ELT(1 + MOD(n, 6), 'wireless','ergonomic','stainless','portable','organic','premium'),
              ' item designed for everyday use. ',
              ELT(1 + MOD(n, 4), 'Lightweight and durable.', 'Comes with a two-year warranty.',
                                 'Eco-friendly packaging.', 'Best seller in its class.')),
       ROUND(5 + 495 * POW(rnd('pp', n), 2), 2),           -- many cheap items, few expensive
       FLOOR(rnd('sk', n) * 500),
       DATE_ADD('2022-06-01', INTERVAL FLOOR(rnd('pc', n) * 200) DAY)
FROM seq WHERE n <= 2000
```

OUTPUT

```
Query OK, 2000 rows affected (0.08 sec)
```

*50,000 orders. * customer picked with a mild skew (some heavy buyers, many light ones) * order date = signup date + exponential delay (mean ~120 days) -> retention decays over time * status: 70% delivered / 15% shipped / 10% returned / 5% cancelled -> LOW-SELECTIVITY column*

SQL

```
INSERT INTO orders (customer_id, order_date, status)
SELECT c.customer_id,
       DATE_ADD(c.signup_date, INTERVAL LEAST(600, FLOOR(-LN(1 - rnd('od', s.n)) * 120)) DAY),
       CASE WHEN rnd('st', s.n) < 0.70 THEN 'delivered'
            WHEN rnd('st', s.n) < 0.85 THEN 'shipped'
            WHEN rnd('st', s.n) < 0.95 THEN 'returned'
            ELSE 'cancelled' END
FROM seq s
JOIN customers c ON c.customer_id = 1 + FLOOR(5000 * POW(rnd('cu', s.n), 1.25))
WHERE s.n <= 50000
ORDER BY s.n
```

OUTPUT

```
Query OK, 50000 rows affected (1.61 sec)
```

100,000 line items (2 per order). Popular products get more sales (skew toward low ids). In "returned" orders, each item is returned with a seller-dependent probability (30%-100%).

SQL

```
INSERT INTO order_items (order_id, product_id, seller_id, quantity, unit_price, returned)
SELECT o.order_id, p.product_id, p.seller_id, 1 + FLOOR(3 * rnd('qt', s.n)), p.price,
       (o.status = 'returned' AND rnd('rt', s.n) < LEAST(1, 0.3 + p.seller_id / 60))
FROM seq s
JOIN orders   o ON o.order_id   = 1 + MOD(s.n - 1, 50000)
JOIN products p ON p.product_id = 1 + FLOOR(2000 * POW(rnd('pr', s.n), 1.5))
WHERE s.n <= 100000
ORDER BY s.n
```

OUTPUT

```
Query OK, 100000 rows affected (3.74 sec)
```

SQL

```
UPDATE orders o
JOIN (SELECT order_id, SUM(quantity * unit_price) AS t FROM order_items GROUP BY order_id) x
  ON x.order_id = o.order_id
SET o.total_amount = x.t
```

OUTPUT

```
Query OK, 50000 rows affected (0.53 sec)
```

~10,000 review attempts (duplicates of the same customer+product are skipped by IGNORE). Each product has a built-in "quality" so some products really are better rated than others.

SQL

```
INSERT IGNORE INTO reviews (product_id, customer_id, rating, title, body, created_at)
SELECT t.pid, t.cid,
       GREATEST(1, LEAST(5, ROUND(2.2 + 2.6 * (MOD(t.pid * 7, 10) / 9) + (rnd('rr', t.n) - 0.5) * 2.4))),
       CONCAT('Review ', t.n),
       'Sample review text.',
       DATE_ADD('2023-03-01', INTERVAL FLOOR(rnd('rd', t.n) * 900) DAY)
FROM (SELECT n,
             1 + FLOOR(2000 * POW(rnd('rp', n), 1.3)) AS pid,
             1 + FLOOR(5000 * rnd('rc', n))           AS cid
      FROM seq WHERE n <= 10000) t
```

OUTPUT

```
Query OK, 9994 rows affected (0.63 sec)
```

SQL

```
ANALYZE TABLE categories, sellers, customers, products, orders, order_items, reviews
```

OUTPUT

```
+------------------+---------+----------+----------+
| Table            | Op      | Msg_type | Msg_text |
+------------------+---------+----------+----------+
| shop.categories  | analyze | status   | OK       |
| shop.sellers     | analyze | status   | OK       |
| shop.customers   | analyze | status   | OK       |
| shop.products    | analyze | status   | OK       |
| shop.orders      | analyze | status   | OK       |
| shop.order_items | analyze | status   | OK       |
| shop.reviews     | analyze | status   | OK       |
+------------------+---------+----------+----------+
7 rows in set (0.03 sec)
```

```
/* Base view reused everywhere: one row per sold line, cancelled orders excluded.
   net_revenue = 0 for returned lines. */
```

SQL

```
CREATE OR REPLACE VIEW v_sales_lines AS
SELECT oi.order_item_id, o.order_id, o.customer_id, o.order_date,
       DATE_SUB(o.order_date, INTERVAL DAYOFMONTH(o.order_date) - 1 DAY) AS order_month,
       oi.product_id, p.category_id, oi.seller_id, oi.quantity,
       oi.quantity * oi.unit_price                        AS gross_revenue,
       IF(oi.returned = 1, 0, oi.quantity * oi.unit_price) AS net_revenue,
       oi.returned
FROM order_items oi
JOIN orders   o ON o.order_id   = oi.order_id
JOIN products p ON p.product_id = oi.product_id
WHERE o.status <> 'cancelled'
```

OUTPUT

```
Query OK, 0 rows affected (0.00 sec)
```

3. Sales trends, top sellers and customer satisfaction

Monthly revenue, units and revenue per order show the sales trend. The best-selling product by net revenue is **Organic Keyboard 10** (Sports) with 169,572. The peak month is 2024-10 with 1,301,011 net revenue. (The dataset is synthetic and has no new signups after late 2024, so revenue later in the timeline fades; that is an artefact of the generated data.)

Customer satisfaction is measured per product from 9,994 reviews (overall average 3.47): average rating, percentage positive (4-5 stars), an NPS-style satisfaction index (positive% minus negative%), and a **Bayesian-weighted rating** that pulls products with few reviews toward the global mean, so a product with three 5-star reviews cannot outrank one with fifty 4.8-star reviews.

3a. Monthly sales trend

SQL

```
SELECT order_month,
       COUNT(DISTINCT order_id)                                AS orders,
       SUM(quantity)                                           AS units,
       ROUND(SUM(net_revenue), 2)                              AS net_revenue,
       ROUND(SUM(net_revenue) / COUNT(DISTINCT order_id), 2)   AS revenue_per_order
FROM v_sales_lines
GROUP BY order_month
ORDER BY order_month
```

OUTPUT

```
+-------------+--------+-------+-------------+------------------+
| order_month | orders | units | net_revenue | revenue_per_order |
+-------------+--------+-------+-------------+------------------+
| 2023-01-01  |    272 |  1070 |   170717.57 |            627.64 |
| 2023-02-01  |    576 |  2314 |   376445.68 |            653.55 |
| 2023-03-01  |   1039 |  4115 |   642341.72 |            618.23 |
| 2023-04-01  |   1202 |  4821 |   742659.61 |            617.85 |
| 2023-05-01  |   1437 |  5823 |   881485.16 |            613.42 |
| 2023-06-01  |   1523 |  6107 |   941819.45 |            618.40 |
| 2023-07-01  |   1764 |  7037 |  1105736.59 |            626.83 |
| 2023-08-01  |   1799 |  7261 |  1106469.77 |            615.05 |
| 2023-09-01  |   1817 |  7331 |  1112678.05 |            612.37 |
| 2023-10-01  |   1834 |  7377 |  1138624.62 |            620.84 |
| 2023-11-01  |   1928 |  7693 |  1209326.59 |            627.24 |
| 2023-12-01  |   2072 |  8326 |  1265656.76 |            610.84 |
| 2024-01-01  |   2051 |  8205 |  1254797.08 |            611.80 |
| 2024-02-01  |   1884 |  7590 |  1169555.78 |            620.78 |
... (29 more rows - full output in outputs/ folder of the repo)
+-------------+--------+-------+-------------+------------------+
43 rows in set (0.38 sec)
```

3b. Top 10 products by net revenue

SQL

```
SELECT p.product_id, p.title, c.name AS category,
       SUM(l.quantity)               AS units_sold,
       ROUND(SUM(l.net_revenue), 2)  AS net_revenue,
       ROUND(100 * AVG(l.returned), 1) AS return_rate_pct
FROM v_sales_lines l
JOIN products   p ON p.product_id  = l.product_id
JOIN categories c ON c.category_id = p.category_id
GROUP BY p.product_id, p.title, c.name
ORDER BY net_revenue DESC
LIMIT 10
```

OUTPUT

```
+------------+------------------------+----------------+------------+-------------+-----------------+
| product_id | title                  | category       | units_sold | net_revenue | return_rate_pct |
+------------+------------------------+----------------+------------+-------------+-----------------+
|         10 | Organic Keyboard 10    | Sports         |        398 |   169572.18 |             4.0 |
|          2 | Stainless Headphones 2 | Sports         |        672 |   138205.26 |             8.0 |
|         17 | Premium Bottle 17      | Home & Kitchen |        291 |   121156.75 |             4.6 |
|          7 | Ergonomic Keyboard 7   | Garden         |        483 |   117317.02 |            10.7 |
|         35 | Premium Backpack 35    | Books          |        274 |   114349.58 |             6.6 |
|         20 | Stainless Speaker 20   | Beauty         |        285 |    96699.76 |             6.1 |
|         79 | Ergonomic Keyboard 79  | Garden         |        222 |    93852.00 |             7.5 |
|          6 | Wireless Keyboard 6    | Fashion        |        437 |    92632.32 |             6.6 |
|         34 | Organic Backpack 34    | Sports         |        252 |    87865.38 |            12.6 |
|         53 | Premium Bottle 53      | Toys           |        207 |    84634.36 |             8.9 |
+------------+------------------------+----------------+------------+-------------+-----------------+
10 rows in set (0.42 sec)
```

3c. Customer satisfaction per product positive% = share of 4-5 stars, negative% = share of 1-2 stars satisfaction_index = positive% - negative% (NPS-style, range -100..100) bayes_rating shrinks products with few reviews toward the global mean (m = 10 pseudo-reviews), so one lucky 5-star review can't top the chart.

SQL

```
SELECT p.product_id, p.title,
       COUNT(*)                                        AS review_count,
       ROUND(AVG(r.rating), 2)                         AS avg_rating,
       ROUND(100 * AVG(r.rating >= 4), 1)              AS positive_pct,
       ROUND(100 * AVG(r.rating <= 2), 1)              AS negative_pct,
       ROUND(100 * (AVG(r.rating >= 4) - AVG(r.rating <= 2)), 1) AS satisfaction_index,
       ROUND((COUNT(*) / (COUNT(*) + 10)) * AVG(r.rating)
            + (10 / (COUNT(*) + 10)) * g.global_avg, 2)    AS bayes_rating
FROM reviews r
JOIN products p ON p.product_id = r.product_id
CROSS JOIN (SELECT AVG(rating) AS global_avg FROM reviews) g
GROUP BY p.product_id, p.title, g.global_avg
HAVING COUNT(*) >= 3
ORDER BY bayes_rating DESC
LIMIT 15
```

OUTPUT

```
+------------+-------------------------+--------------+------------+--------------+--------------+--------------------+--------------+
| product_id | title                   | review_count | avg_rating | positive_pct | negative_pct | satisfaction_index | bayes_rating |
+------------+-------------------------+--------------+------------+--------------+--------------+--------------------+--------------+
|         27 | Portable Lamp 27        |           11 |       4.82 |        100.0 |          0.0 |              100.0 |         4.18 |
|          1 | Ergonomic Headphones 1  |           26 |       4.42 |         88.5 |          0.0 |               88.5 |         4.16 |
|         37 | Ergonomic Headphones 37 |           10 |       4.80 |        100.0 |          0.0 |              100.0 |         4.14 |
|        397 | Ergonomic Headphones 397|            9 |       4.89 |        100.0 |          0.0 |              100.0 |         4.14 |
|        124 | Organic Bottle 124      |           11 |       4.73 |         90.9 |          0.0 |               90.9 |         4.13 |
|        127 | Ergonomic Speaker 127   |           11 |       4.73 |        100.0 |          0.0 |              100.0 |         4.13 |
|        377 | Premium Bottle 377      |           11 |       4.73 |        100.0 |          0.0 |              100.0 |         4.13 |
|         34 | Organic Backpack 34     |           12 |       4.67 |        100.0 |          0.0 |              100.0 |         4.12 |
|         67 | Ergonomic Backpack 67   |            7 |       5.00 |        100.0 |          0.0 |              100.0 |         4.10 |
|       1527 | Portable Bottle 1527    |            7 |       5.00 |        100.0 |          0.0 |              100.0 |         4.10 |
|       1617 | Portable Backpack 1617  |            7 |       5.00 |        100.0 |          0.0 |              100.0 |         4.10 |
|         47 | Premium Keyboard 47     |            9 |       4.78 |        100.0 |          0.0 |              100.0 |         4.09 |
|        157 | Ergonomic Bottle 157    |            8 |       4.88 |        100.0 |          0.0 |              100.0 |         4.09 |
|        917 | Premium Bottle 917      |            9 |       4.78 |        100.0 |          0.0 |              100.0 |         4.09 |
|        997 | Ergonomic Lamp 997      |            9 |       4.78 |        100.0 |          0.0 |              100.0 |         4.09 |
+------------+-------------------------+--------------+------------+--------------+--------------+--------------------+--------------+
15 rows in set (0.04 sec)
```

3d. Satisfaction by category

SQL

```
SELECT c.name AS category,
       COUNT(*) AS reviews,
       ROUND(AVG(r.rating), 2) AS avg_rating,
       ROUND(100 * AVG(r.rating >= 4), 1) AS positive_pct
FROM reviews r
JOIN products p   ON p.product_id  = r.product_id
JOIN categories c ON c.category_id = p.category_id
GROUP BY c.name
ORDER BY avg_rating DESC
```

OUTPUT

```
+----------------+---------+------------+--------------+
| category       | reviews | avg_rating | positive_pct |
+----------------+---------+------------+--------------+
| Home & Kitchen |    1220 |       3.60 |         55.1 |
| Toys           |    1222 |       3.60 |         54.7 |
| Books          |    1287 |       3.59 |         54.9 |
| Garden         |    1244 |       3.58 |         54.6 |
| Sports         |    1263 |       3.37 |         46.4 |
| Electronics    |    1260 |       3.35 |         46.4 |
| Beauty         |    1283 |       3.34 |         44.5 |
| Fashion        |    1215 |       3.34 |         46.4 |
+----------------+---------+------------+--------------+
8 rows in set (0.02 sec)
```

4. Cohort analysis

Customers are grouped by the month of their first non-cancelled purchase. For cohorts with at least 100 customers (up to Dec 2025), average retention is **79.7%** one month later, **61.9%** after three months and **37.6%** after six months.

How this improves marketing:

- A large drop after month 0 means the first 30 days matter most: trigger a second-purchase offer or onboarding email in days 7-21.
- Compare cohorts: a cohort with better month-3 retention points to an acquisition channel or campaign worth scaling.
- Cumulative revenue per customer shows how quickly each cohort pays back acquisition cost.
- Days-to-second-order tells you when to send a win-back message: just before the typical gap ends.

SQL

```
CREATE OR REPLACE VIEW v_customer_cohort_activity AS
SELECT customer_id, order_month, cohort_month,
       TIMESTAMPDIFF(MONTH, cohort_month, order_month) AS month_offset,
       revenue
FROM (
  SELECT customer_id, order_month, revenue,
         MIN(order_month) OVER (PARTITION BY customer_id) AS cohort_month
  FROM (
    SELECT customer_id, order_month, SUM(net_revenue) AS revenue
    FROM v_sales_lines
    GROUP BY customer_id, order_month
  ) m
) x
```

OUTPUT

```
Query OK, 0 rows affected (0.01 sec)
```

4a. Retention matrix (long form): % of cohort still buying N months later

SQL

```
SELECT cohort_month, month_offset, active_customers,
       ROUND(100 * active_customers /
             FIRST_VALUE(active_customers) OVER (PARTITION BY cohort_month ORDER BY month_offset), 1)
         AS retention_pct
FROM (
  SELECT cohort_month, month_offset, COUNT(*) AS active_customers
  FROM v_customer_cohort_activity
  GROUP BY cohort_month, month_offset
) r
ORDER BY cohort_month, month_offset
```

OUTPUT

```
+--------------+--------------+------------------+--------------+
| cohort_month | month_offset | active_customers | retention_pct |
+--------------+--------------+------------------+--------------+
| 2023-01-01   |            0 |              137 |         100.0 |
| 2023-01-01   |            1 |              112 |          81.8 |
| 2023-01-01   |            2 |              109 |          79.6 |
| 2023-01-01   |            3 |               91 |          66.4 |
| 2023-01-01   |            4 |               73 |          53.3 |
| 2023-01-01   |            5 |               72 |          52.6 |
| 2023-01-01   |            6 |               63 |          46.0 |
| 2023-01-01   |            7 |               40 |          29.2 |
| 2023-01-01   |            8 |               43 |          31.4 |
| 2023-01-01   |            9 |               29 |          21.2 |
| 2023-01-01   |           10 |               23 |          16.8 |
| 2023-01-01   |           11 |               17 |          12.4 |
... (514 more rows - full output in outputs/ folder of the repo)
+--------------+--------------+------------------+--------------+
526 rows in set (0.43 sec)
```

4b. Retention matrix (pivoted, first 6 months) - ready for a heatmap

SQL

```
SELECT cohort_month,
       MAX(CASE WHEN month_offset = 0 THEN active END) AS cohort_size,
       ROUND(100 * MAX(CASE WHEN month_offset = 1 THEN active END) / MAX(CASE WHEN month_offset = 0 THEN active END), 1) AS m1,
       ROUND(100 * MAX(CASE WHEN month_offset = 2 THEN active END) / MAX(CASE WHEN month_offset = 0 THEN active END), 1) AS m2,
       ROUND(100 * MAX(CASE WHEN month_offset = 3 THEN active END) / MAX(CASE WHEN month_offset = 0 THEN active END), 1) AS m3,
       ROUND(100 * MAX(CASE WHEN month_offset = 4 THEN active END) / MAX(CASE WHEN month_offset = 0 THEN active END), 1) AS m4,
       ROUND(100 * MAX(CASE WHEN month_offset = 5 THEN active END) / MAX(CASE WHEN month_offset = 0 THEN active END), 1) AS m5,
       ROUND(100 * MAX(CASE WHEN month_offset = 6 THEN active END) / MAX(CASE WHEN month_offset = 0 THEN active END), 1) AS m6
FROM (
  SELECT cohort_month, month_offset, COUNT(*) AS active
  FROM v_customer_cohort_activity
  GROUP BY cohort_month, month_offset
) t
GROUP BY cohort_month
ORDER BY cohort_month
```

OUTPUT

```
+--------------+-------------+------+-------+-------+------+------+------+
| cohort_month | cohort_size | m1   | m2    | m3    | m4   | m5   | m6   |
+--------------+-------------+------+-------+-------+------+------+------+
| 2023-01-01   |         137 | 81.8 |  79.6 |  66.4 | 53.3 | 52.6 | 46.0 |
| 2023-02-01   |         179 | 84.4 |  73.7 |  66.5 | 49.7 | 50.3 | 35.8 |
| 2023-03-01   |         216 | 78.7 |  71.8 |  62.5 | 51.9 | 46.3 | 36.6 |
| 2023-04-01   |         222 | 83.3 |  66.7 |  62.6 | 47.7 | 42.8 | 35.6 |
| 2023-05-01   |         240 | 82.1 |  74.2 |  61.7 | 52.9 | 47.1 | 34.6 |
| 2023-06-01   |         213 | 81.2 |  68.5 |  57.3 | 51.6 | 43.2 | 37.6 |
| 2023-07-01   |         214 | 76.6 |  72.0 |  58.9 | 60.3 | 41.1 | 36.4 |
| 2023-08-01   |         230 | 77.4 |  70.4 |  64.8 | 60.0 | 45.7 | 32.2 |
| 2023-09-01   |         212 | 73.1 |  72.2 |  64.2 | 56.6 | 46.2 | 39.6 |
| 2023-10-01   |         211 | 76.8 |  76.3 |  67.8 | 53.6 | 46.0 | 32.2 |
| 2023-11-01   |         234 | 79.9 |  70.9 |  57.3 | 56.4 | 39.7 | 38.9 |
| 2023-12-01   |         226 | 80.1 |  65.9 |  60.6 | 54.0 | 43.8 | 32.3 |
... (16 more rows - full output in outputs/ folder of the repo)
+--------------+-------------+------+-------+-------+------+------+------+
28 rows in set (0.36 sec)
```

4c. Cumulative revenue per customer by cohort age (how fast does a cohort pay back?)

SQL

```
WITH sizes AS (
  SELECT cohort_month, COUNT(*) AS cohort_size
  FROM v_customer_cohort_activity WHERE month_offset = 0 GROUP BY cohort_month
), rev AS (
  SELECT cohort_month, month_offset, SUM(revenue) AS revenue
  FROM v_customer_cohort_activity GROUP BY cohort_month, month_offset
)
SELECT r.cohort_month, r.month_offset,
       ROUND(SUM(r.revenue) OVER (PARTITION BY r.cohort_month ORDER BY r.month_offset) / s.cohort_size, 2)
         AS cumulative_revenue_per_customer
FROM rev r JOIN sizes s USING (cohort_month)
ORDER BY r.cohort_month, r.month_offset
```

OUTPUT

```
+--------------+--------------+---------------------------------+
| cohort_month | month_offset | cumulative_revenue_per_customer |
+--------------+--------------+---------------------------------+
| 2023-01-01   |            0 |                         1246.11 |
| 2023-01-01   |            1 |                         2441.61 |
| 2023-01-01   |            2 |                         3555.37 |
| 2023-01-01   |            3 |                         4266.05 |
| 2023-01-01   |            4 |                         4718.07 |
| 2023-01-01   |            5 |                         5081.92 |
| 2023-01-01   |            6 |                         5407.69 |
| 2023-01-01   |            7 |                         5607.87 |
| 2023-01-01   |            8 |                         5846.05 |
| 2023-01-01   |            9 |                         5991.38 |
| 2023-01-01   |           10 |                         6119.19 |
| 2023-01-01   |           11 |                         6213.89 |
... (514 more rows - full output in outputs/ folder of the repo)
+--------------+--------------+---------------------------------+
526 rows in set (0.73 sec)
```

4d. Days from first to second order, by cohort (is a re-engagement nudge needed?)

SQL

```
WITH ranked AS (
  SELECT customer_id, order_date,
         DATE_SUB(MIN(order_date) OVER (PARTITION BY customer_id),
                  INTERVAL DAYOFMONTH(MIN(order_date) OVER (PARTITION BY customer_id)) - 1 DAY) AS cohort_month,
         ROW_NUMBER() OVER (PARTITION BY customer_id ORDER BY order_date, order_id) AS rn,
         LAG(order_date) OVER (PARTITION BY customer_id ORDER BY order_date, order_id) AS prev_date
  FROM orders WHERE status <> 'cancelled'
)
SELECT cohort_month,
       COUNT(*)                                  AS repeat_buyers,
       ROUND(AVG(DATEDIFF(order_date, prev_date)), 1) AS avg_days_to_second_order
FROM ranked WHERE rn = 2
GROUP BY cohort_month
ORDER BY cohort_month
```

OUTPUT

```
+--------------+---------------+--------------------------+
| cohort_month | repeat_buyers | avg_days_to_second_order |
+--------------+---------------+--------------------------+
| 2023-01-01   |           136 |                     13.6 |
| 2023-02-01   |           179 |                     18.2 |
| 2023-03-01   |           216 |                     18.8 |
| 2023-04-01   |           222 |                     15.3 |
| 2023-05-01   |           239 |                     16.1 |
| 2023-06-01   |           213 |                     16.5 |
| 2023-07-01   |           213 |                     21.3 |
| 2023-08-01   |           230 |                     19.4 |
| 2023-09-01   |           212 |                     18.8 |
| 2023-10-01   |           210 |                     18.2 |
| 2023-11-01   |           234 |                     19.3 |
| 2023-12-01   |           226 |                     17.1 |
... (15 more rows - full output in outputs/ folder of the repo)
+--------------+---------------+--------------------------+
27 rows in set (0.28 sec)
```

```
/* Marketing reading guide
   - Steep drop at m1 -> fix onboarding / send a 2nd-purchase offer in days 7-21.
   - Cohorts with higher m3 retention -> find what acquisition channel/campaign they came from, buy more of it.
   - Cumulative revenue per customer flattening -> time win-back campaigns near the flattening month.
   - 4d tells you *when* to nudge: target roughly the average gap, before it lapses. */
```

5. Indexing and EXPLAIN

Composite indexes put the equality column first and the range/sort column second, so MySQL can jump to one customer's orders, read them already sorted by date, and avoid a filesort. **FULLTEXT** replaces leading-wildcard LIKE scans, which can never use a B-tree index. MySQL also dropped the old single-column foreign-key index automatically, because the composite's left prefix covers it (see SHOW INDEX).

Low-selectivity example: orders.status has 4 distinct values over 50,000 rows (selectivity 0.0001). Asking for 'delivered' (about 70% of rows) means the index would visit about 35,000 rows one by one; the EXPLAIN ANALYZE comparison below shows the forced-index plan takes roughly twice as long as a plain table scan. Asking for the rare value 'cancelled' (about 5%) is where an index helps. Rule of thumb: an index pays off when it narrows the result to a small fraction of the table. Note the optimizer's row estimate for the index plan can be optimistic, so always verify with EXPLAIN ANALYZE.

5a. orders (customer_id, order_date) BEFORE: no usable index -> full table scan (type=ALL, rows ~ 50,000)

SQL

```
EXPLAIN
SELECT order_id, order_date, total_amount
FROM orders IGNORE INDEX (fk_orders_customer)
WHERE customer_id = 1234 AND order_date BETWEEN '2024-01-01' AND '2024-06-30'
ORDER BY order_date
```

OUTPUT

```
+----+-------------+--------+------------+------+---------------+------+---------+------+-------+----------+-----------------------------+
| id | select_type | table  | partitions | type | possible_keys | key  | key_len | ref  | rows  | filtered | Extra                       |
+----+-------------+--------+------------+------+---------------+------+---------+------+-------+----------+-----------------------------+
|  1 | SIMPLE      | orders | NULL       | ALL  | NULL          | NULL | NULL    | NULL | 50250 |      0.0 | Using where; Using filesort |
+----+-------------+--------+------------+------+---------------+------+---------+------+-------+----------+-----------------------------+
1 row in set (0.00 sec)
```

Equality column first, range/sort column second.

SQL

```
CREATE INDEX idx_orders_customer_date ON orders (customer_id, order_date)
```

OUTPUT

```
Query OK, 0 rows affected (0.12 sec)
```

AFTER: type=range, key=idx_orders_customer_date, rows ~ a handful, and NO "Using filesort" because the index is already ordered by order_date.

SQL

```
EXPLAIN
SELECT order_id, order_date, total_amount
FROM orders
WHERE customer_id = 1234 AND order_date BETWEEN '2024-01-01' AND '2024-06-30'
ORDER BY order_date
```

OUTPUT

```
+----+-------------+--------+------------+-------+---------------------------+---------------------------+---------+------+------+----------+-----------------------+
| id | select_type | table  | partitions | type  | possible_keys             | key                       | key_len | ref  | rows | filtered | Extra                 |
+----+-------------+--------+------------+-------+---------------------------+---------------------------+---------+------+------+----------+-----------------------+
|  1 | SIMPLE      | orders | NULL       | range | idx_orders_customer_date | idx_orders_customer_date | 7       | NULL |    8 |    100.0 | Using index condition |
+----+-------------+--------+------------+-------+---------------------------+---------------------------+---------+------+------+----------+-----------------------+
1 row in set (0.00 sec)
```

Actual runtime, without vs with the index (read "actual time" in each plan; MySQL 8.0.18+)

SQL

```
EXPLAIN ANALYZE
SELECT order_id, order_date, total_amount
FROM orders IGNORE INDEX (idx_orders_customer_date)
WHERE customer_id = 1234 AND order_date BETWEEN '2024-01-01' AND '2024-06-30'
ORDER BY order_date
```

OUTPUT

```
-> Sort: orders.order_date  (cost=5065 rows=50250) (actual time=11..11 rows=8 loops=1)
    -> Filter: ((orders.customer_id = 1234) and (orders.order_date between '2024-01-01' and '2024-06-30'))  (cost=5065 rows=50250) (actual time=2.33..11 rows=8 loops=1)
        -> Table scan on orders  (cost=5065 rows=50250) (actual time=0.5..8.26 rows=50000 loops=1)
```

SQL

```
EXPLAIN ANALYZE
SELECT order_id, order_date, total_amount
FROM orders
WHERE customer_id = 1234 AND order_date BETWEEN '2024-01-01' AND '2024-06-30'
ORDER BY order_date
```

OUTPUT

```
-> Index range scan on orders using idx_orders_customer_date over (customer_id = 1234 AND '2024-01-01' <= order_date <= '2024-06-30'), with index condition: ((orders.customer_id = 1234)
    and (orders.order_date between '2024-01-01' and '2024-06-30'))  (cost=3.86 rows=8) (actual time=0.037..0.0398 rows=8 loops=1)
```

The composite's left prefix (customer_id) can also serve the foreign key, so MySQL silently drops the old single-column FK index when the composite is created. Verify:

SQL

```
SHOW INDEX FROM orders
```

OUTPUT

```
+--------+------------+--------------------------+--------------+-------------+-----------+-------------+----------+--------+------+------------+---------+---------------+---------+------------+
| Table  | Non_unique | Key_name                 | Seq_in_index | Column_name | Collation | Cardinality | Sub_part | Packed | Null | Index_type | Comment | Index_comment | Visible | Expression |
+--------+------------+--------------------------+--------------+-------------+-----------+-------------+----------+--------+------+------------+---------+---------------+---------+------------+
| orders |          0 | PRIMARY                  |            1 | order_id    | A         |       50250 | NULL     | NULL   |      | BTREE      |         |               | YES     | NULL       |
| orders |          1 | idx_orders_customer_date |            1 | customer_id | A         |        4726 | NULL     | NULL   |      | BTREE      |         |               | YES     | NULL       |
| orders |          1 | idx_orders_customer_date |            2 | order_date  | A         |       49580 | NULL     | NULL   |      | BTREE      |         |               | YES     | NULL       |
+--------+------------+--------------------------+--------------+-------------+-----------+-------------+----------+--------+------+------------+---------+---------------+---------+------------+
3 rows in set (0.00 sec)
```

5b. products (category_id, price)

SQL

```
EXPLAIN
SELECT product_id, title, price
FROM products IGNORE INDEX (fk_products_category)
WHERE category_id = 3 AND price BETWEEN 20 AND 50
ORDER BY price                                  -- type=ALL + Using filesort
```

OUTPUT

```
+----+-------------+----------+------------+------+---------------+------+---------+------+------+----------+-----------------------------+
| id | select_type | table    | partitions | type | possible_keys | key  | key_len | ref  | rows | filtered | Extra                       |
+----+-------------+----------+------------+------+---------------+------+---------+------+------+----------+-----------------------------+
|  1 | SIMPLE      | products | NULL       | ALL  | NULL          | NULL | NULL    | NULL | 2000 |     1.39 | Using where; Using filesort |
+----+-------------+----------+------------+------+---------------+------+---------+------+------+----------+-----------------------------+
1 row in set (0.00 sec)
```

SQL

```
CREATE INDEX idx_products_category_price ON products (category_id, price)
```

OUTPUT

```
Query OK, 0 rows affected (0.03 sec)
```

SQL

```
EXPLAIN
SELECT product_id, title, price
FROM products
WHERE category_id = 3 AND price BETWEEN 20 AND 50
ORDER BY price                                  -- type=range, no filesort
```

OUTPUT

```
+----+-------------+----------+------------+-------+----------------------------+----------------------------+---------+------+------+----------+-----------------------+
| id | select_type | table    | partitions | type  | possible_keys              | key                        | key_len | ref  | rows | filtered | Extra                 |
+----+-------------+----------+------------+-------+----------------------------+----------------------------+---------+------+------+----------+-----------------------+
|  1 | SIMPLE      | products | NULL       | range | idx_products_category_price | idx_products_category_price | 9       | NULL |   41 |    100.0 | Using index condition |
+----+-------------+----------+------------+-------+----------------------------+----------------------------+---------+------+------+----------+-----------------------+
1 row in set (0.00 sec)
```

SQL

```
EXPLAIN ANALYZE
SELECT product_id, title, price
FROM products IGNORE INDEX (idx_products_category_price)
WHERE category_id = 3 AND price BETWEEN 20 AND 50
ORDER BY price
```

OUTPUT

```
-> Sort: products.price  (cost=205 rows=2000) (actual time=0.624..0.626 rows=41 loops=1)
    -> Filter: ((products.category_id = 3) and (products.price between 20 and 50))  (cost=205 rows=2000) (actual time=0.09..0.602 rows=41 loops=1)
        -> Table scan on products  (cost=205 rows=2000) (actual time=0.077..0.442 rows=2000 loops=1)
```

SQL

```
EXPLAIN ANALYZE
SELECT product_id, title, price
FROM products
WHERE category_id = 3 AND price BETWEEN 20 AND 50
ORDER BY price
```

OUTPUT

```
-> Index range scan on products using idx_products_category_price over (category_id = 3 AND 20.00 <= price <= 50.00), with index condition: ((products.category_id = 3) and
    (products.price between 20 and 50))  (cost=18.7 rows=41) (actual time=0.022..0.0872 rows=41 loops=1)
```

5c. FULLTEXT on title + description BEFORE: leading-wildcard LIKE can never use a B-tree index -> full scan

SQL

```
EXPLAIN
SELECT product_id, title FROM products
WHERE title LIKE '%wireless%' OR description LIKE '%wireless%'
```

OUTPUT

```
+----+-------------+----------+------------+------+---------------+------+---------+------+------+----------+-------------+
| id | select_type | table    | partitions | type | possible_keys | key  | key_len | ref  | rows | filtered | Extra       |
+----+-------------+----------+------------+------+---------------+------+---------+------+------+----------+-------------+
|  1 | SIMPLE      | products | NULL       | ALL  | NULL          | NULL | NULL    | NULL | 2000 |    20.99 | Using where |
+----+-------------+----------+------------+------+---------------+------+---------+------+------+----------+-------------+
1 row in set (0.00 sec)
```

SQL

```
ALTER TABLE products ADD FULLTEXT INDEX ft_products_title_desc (title, description)
```

OUTPUT

```
Query OK, 0 rows affected (0.15 sec)
```

AFTER: type=fulltext, key=ft_products_title_desc

SQL

```
EXPLAIN
SELECT product_id, title
FROM products
WHERE MATCH(title, description) AGAINST ('wireless headphones' IN NATURAL LANGUAGE MODE)
```

OUTPUT

```
+----+-------------+----------+------------+----------+-----------------------+-----------------------+---------+-------+------+----------+-------------------------------+
| id | select_type | table    | partitions | type     | possible_keys         | key                   | key_len | ref   | rows | filtered | Extra                         |
+----+-------------+----------+------------+----------+-----------------------+-----------------------+---------+-------+------+----------+-------------------------------+
|  1 | SIMPLE      | products | NULL       | fulltext | ft_products_title_desc | ft_products_title_desc | 0       | const |    1 |    100.0 | Using where; Ft_hints: sorted |
+----+-------------+----------+------------+----------+-----------------------+-----------------------+---------+-------+------+----------+-------------------------------+
1 row in set (0.00 sec)
```

*Ranked search with relevance score; BOOLEAN MODE: + required, - excluded, * prefix*

SQL

```
SELECT product_id, title,
       MATCH(title, description) AGAINST ('wireless headphones' IN NATURAL LANGUAGE MODE) AS relevance
FROM products
WHERE MATCH(title, description) AGAINST ('+wireless -backpack ergonom*' IN BOOLEAN MODE)
ORDER BY relevance DESC
LIMIT 10
```

OUTPUT

```
+------------+--------------------------+-------------------+
| product_id | title                    | relevance         |
+------------+--------------------------+-------------------+
|       1296 | Wireless Headphones 1296 | 1.814544677734375 |
|        468 | Wireless Headphones 468  | 1.814544677734375 |
|        828 | Wireless Headphones 828  | 1.814544677734375 |
|       1656 | Wireless Headphones 1656 | 1.814544677734375 |
|        432 | Wireless Headphones 432  | 1.814544677734375 |
|       1332 | Wireless Headphones 1332 | 1.814544677734375 |
|        864 | Wireless Headphones 864  | 1.814544677734375 |
|       1692 | Wireless Headphones 1692 | 1.814544677734375 |
|        396 | Wireless Headphones 396  | 1.814544677734375 |
|       1260 | Wireless Headphones 1260 | 1.814544677734375 |
+------------+--------------------------+-------------------+
10 rows in set (0.00 sec)
```

Caveats: InnoDB ignores words shorter than 3 chars (innodb_ft_min_token_size) and stopwords; new rows become searchable at commit; LIKE is still needed for substring matches.

5d. Why an index on a low-selectivity column barely helps Selectivity = distinct values / rows. For status it is ~ 4/50000 = 0.0001.

SQL

```
SELECT COUNT(DISTINCT status) / COUNT(*) AS selectivity FROM orders
```

OUTPUT

```
+-------------+
| selectivity |
+-------------+
|      0.0001 |
+-------------+
1 row in set (0.01 sec)
```

SQL

```
SELECT status, COUNT(*) AS cnt, ROUND(100 * COUNT(*) / SUM(COUNT(*)) OVER (), 1) AS pct
FROM orders GROUP BY status                     -- delivered ~70%, cancelled ~4%
```

OUTPUT

```
+-----------+-------+------+
| status    | cnt   | pct  |
+-----------+-------+------+
| returned  |  5027 | 10.1 |
| delivered | 34843 | 69.7 |
| shipped   |  7651 | 15.3 |
| cancelled |  2479 |  5.0 |
+-----------+-------+------+
4 rows in set (0.02 sec)
```

SQL

```
CREATE INDEX idx_orders_status ON orders (status)
```

OUTPUT

```
Query OK, 0 rows affected (0.10 sec)
```

SQL

```
ANALYZE TABLE orders
```

OUTPUT

```
+-------------+---------+----------+----------+
| Table       | Op      | Msg_type | Msg_text |
+-------------+---------+----------+----------+
| shop.orders | analyze | status   | OK       |
+-------------+---------+----------+----------+
1 row in set (0.01 sec)
```

70% of the table. MySQL may still pick the index here (its row estimate is rough), but that does not make it a good plan - compare the ACTUAL times below.

SQL

```
EXPLAIN SELECT order_id, total_amount FROM orders WHERE status = 'delivered'
```

OUTPUT

```
+----+-------------+--------+------------+------+-------------------+-------------------+---------+-------+-------+----------+-----------------------+
| id | select_type | table  | partitions | type | possible_keys     | key               | key_len | ref   | rows  | filtered | Extra                 |
+----+-------------+--------+------------+------+-------------------+-------------------+---------+-------+-------+----------+-----------------------+
|  1 | SIMPLE      | orders | NULL       | ref  | idx_orders_status | idx_orders_status | 1       | const | 25125 |    100.0 | Using index condition |
+----+-------------+--------+------------+------+-------------------+-------------------+---------+-------+-------+----------+-----------------------+
1 row in set (0.00 sec)
```

Same query, once without the index (full scan) and once forcing it (MySQL 8.0.18+). The index path does ~35k random lookups into the clustered index (total_amount is not in the index), so it is typically SLOWER than one sequential scan. Check "actual time" in each plan.

SQL

```
EXPLAIN ANALYZE SELECT order_id, total_amount FROM orders IGNORE INDEX (idx_orders_status)
WHERE status = 'delivered'
```

OUTPUT

```
-> Filter: (orders.`status` = 'delivered')  (cost=5065 rows=16750) (actual time=0.496..14.4 rows=34843 loops=1)
    -> Table scan on orders  (cost=5065 rows=50250) (actual time=0.492..8.62 rows=50000 loops=1)
```

SQL

```
EXPLAIN ANALYZE SELECT order_id, total_amount FROM orders FORCE  INDEX (idx_orders_status)
WHERE status = 'delivered'
```

OUTPUT

```
-> Index lookup on orders using idx_orders_status (status='delivered'), with index condition: (orders.`status` = 'delivered')  (cost=1796 rows=16750) (actual time=1.41..30.4 rows=34843
    loops=1)
```

~4% of the table: now the index is worth using.

SQL

```
EXPLAIN SELECT order_id, total_amount FROM orders WHERE status = 'cancelled'
```

OUTPUT

```
+----+-------------+--------+------------+------+-------------------+-------------------+---------+-------+------+----------+-----------------------+
| id | select_type | table  | partitions | type | possible_keys     | key               | key_len | ref   | rows | filtered | Extra                 |
+----+-------------+--------+------------+------+-------------------+-------------------+---------+-------+------+----------+-----------------------+
|  1 | SIMPLE      | orders | NULL       | ref  | idx_orders_status | idx_orders_status | 1       | const | 2479 |    100.0 | Using index condition |
+----+-------------+--------+------------+------+-------------------+-------------------+---------+-------+------+----------+-----------------------+
1 row in set (0.00 sec)
```

*Takeaways: (1) an index pays off when it narrows to a small fraction of rows (rule of thumb < ~10-20%); the optimizer's estimate can be wrong, so verify with EXPLAIN ANALYZE; (2) low-selectivity columns are useful only as the *trailing/leading part of a composite* (e.g. (status, order_date)) or when queries target the rare values; (3) every extra index slows INSERT/UPDATE and uses space.*

SQL

```
DROP INDEX idx_orders_status ON orders
```

OUTPUT

```
Query OK, 0 rows affected (0.01 sec)
```

Timing evidence (averages of 30 runs)

Query	Without optimisation	With optimisation	Speed-up
Orders of one customer in a date range (composite index customer_id, order_date)	8.94 ms	0.20 ms	44x

Query	Without optimisation	With optimisation	Speed-up
Products in a category within a price range (composite index category_id, price)	0.92 ms	0.32 ms	2.8x
Keyword search 'wireless' (LIKE '%...%' vs FULLTEXT) - only 2,000 rows, so gap is small	2.54 ms	1.71 ms	1.5x
Top 10 sellers: view v_seller_performance vs summary table	289.1 ms	0.31 ms	943x

Averages of 30 runs on MySQL 8.0 inside the same session (warm cache). Absolute times depend on the machine; the point is the relative difference.

6. Views and simulated materialized view

v_seller_performance reports total sales (net of returns), average rating and return rate per seller; **v_customer_ltv** reports lifetime value, order count, average order value and activity span per customer. Sales and reviews are aggregated in separate subqueries to avoid join fan-out (which would otherwise multiply sales by the number of reviews).

MySQL has no native materialized view, so **seller_performance_summary** is a real table filled by the stored procedure `sp_refresh_seller_performance()`, which builds a fresh table and swaps it in with one atomic RENAME so readers never see a half-filled table. An event re-runs it every hour. Reading the top-10 sellers took about 289 ms from the view and 0.31 ms from the summary table.

6a. Seller performance

SQL

```
CREATE OR REPLACE VIEW v_seller_performance AS
SELECT s.seller_id, s.name AS seller_name,
       COALESCE(sa.total_sales, 0)        AS total_sales,       -- net of returns, excl. cancelled
       COALESCE(sa.units_sold, 0)         AS units_sold,
       rv.avg_rating,
       COALESCE(rv.review_count, 0)       AS review_count,
       COALESCE(sa.return_rate_pct, 0)    AS return_rate_pct
FROM sellers s
LEFT JOIN (
    SELECT seller_id,
           SUM(net_revenue)           AS total_sales,
           SUM(quantity)              AS units_sold,
           ROUND(100 * AVG(returned), 2) AS return_rate_pct
    FROM v_sales_lines
    GROUP BY seller_id
) sa ON sa.seller_id = s.seller_id
LEFT JOIN (                                  -- separate subquery avoids join fan-out
    SELECT p.seller_id, ROUND(AVG(r.rating), 2) AS avg_rating, COUNT(*) AS review_count
    FROM reviews r JOIN products p ON p.product_id = r.product_id
    GROUP BY p.seller_id
) rv ON rv.seller_id = s.seller_id
```

OUTPUT

```
Query OK, 0 rows affected (0.00 sec)
```

SQL

```
SELECT * FROM v_seller_performance ORDER BY total_sales DESC LIMIT 10
```

OUTPUT

```
+-----------+-------------+-------------+------------+------------+--------------+-----------------+
| seller_id | seller_name | total_sales | units_sold | avg_rating | review_count | return_rate_pct |
+-----------+-------------+-------------+------------+------------+--------------+-----------------+
|         1 | Seller 1    |  1969694.93 |      12937 |       3.41 |          655 |            3.40 |
|         4 | Seller 4    |  1090679.23 |       6034 |       3.35 |          332 |            3.67 |
|        18 | Seller 18   |  1021465.41 |       5270 |       3.45 |          269 |            6.54 |
|         3 | Seller 3    |  1002840.80 |       6708 |       3.51 |          349 |            3.94 |
|         7 | Seller 7    |  1001377.59 |       6011 |       3.39 |          307 |            4.10 |
|         2 | Seller 2    |   988381.98 |       6977 |       3.34 |          375 |            3.14 |
|         5 | Seller 5    |   915601.64 |       4814 |       3.35 |          236 |            4.49 |
|         6 | Seller 6    |   763962.58 |       5223 |       3.59 |          259 |            4.28 |
|        21 | Seller 21   |   712005.52 |       4787 |       3.34 |          268 |            7.38 |
|        44 | Seller 44   |   709452.17 |       4097 |       3.49 |          185 |           10.02 |
+-----------+-------------+-------------+------------+------------+--------------+-----------------+
10 rows in set (0.27 sec)
```

6b. Customer lifetime value

SQL

```
CREATE OR REPLACE VIEW v_customer_ltv AS
SELECT c.customer_id, c.name, c.signup_date,
       COUNT(DISTINCT l.order_id)                                        AS orders_count,
       MIN(l.order_date)                                                 AS first_order_date,
       MAX(l.order_date)                                                 AS last_order_date,
       DATEDIFF(MAX(l.order_date), MIN(l.order_date))                    AS active_days,
       COALESCE(SUM(l.net_revenue), 0)                                   AS lifetime_value,
       ROUND(COALESCE(SUM(l.net_revenue) / NULLIF(COUNT(DISTINCT l.order_id), 0), 0), 2) AS avg_order_value,
       ROUND(100 * COALESCE(AVG(l.returned), 0), 2)                      AS return_rate_pct
FROM customers c
LEFT JOIN v_sales_lines l ON l.customer_id = c.customer_id
GROUP BY c.customer_id, c.name, c.signup_date
```

OUTPUT

```
Query OK, 0 rows affected (0.00 sec)
```

SQL

```
SELECT * FROM v_customer_ltv ORDER BY lifetime_value DESC LIMIT 10
```

OUTPUT

```
+-------------+-------------+-------------+--------------+------------------+-----------------+-------------+----------------+-----------------+-----------------+
| customer_id | name        | signup_date | orders_count | first_order_date | last_order_date | active_days | lifetime_value | avg_order_value | return_rate_pct |
+-------------+-------------+-------------+--------------+------------------+-----------------+-------------+----------------+-----------------+-----------------+
|           1 | Customer 1  | 2023-06-22  |           57 | 2023-06-22       | 2025-02-11      |         600 |       31058.47 |          544.89 |            8.77 |
|           2 | Customer 2  | 2023-02-01  |           39 | 2023-02-12       | 2024-09-23      |         589 |       27067.10 |          694.03 |            6.41 |
|           3 | Customer 3  | 2023-07-06  |           40 | 2023-07-06       | 2025-02-25      |         600 |       26792.57 |          669.81 |            8.75 |
|          19 | Customer 19 | 2024-09-07  |           39 | 2024-09-08       | 2026-02-04      |         514 |       24057.07 |          616.85 |           11.54 |
|           8 | Customer 8  | 2023-01-20  |           35 | 2023-01-22       | 2024-09-11      |         598 |       23727.73 |          677.94 |           10.00 |
|          74 | Customer 74 | 2024-05-28  |           28 | 2024-05-30       | 2025-09-08      |         466 |       23622.37 |          843.66 |            1.79 |
|          22 | Customer 22 | 2024-07-24  |           34 | 2024-07-25       | 2026-03-08      |         591 |       23184.13 |          681.89 |            7.35 |
|           6 | Customer 6  | 2023-10-29  |           34 | 2023-10-29       | 2025-01-09      |         438 |       22942.39 |          674.78 |            1.47 |
|          14 | Customer 14 | 2023-12-16  |           34 | 2023-12-17       | 2024-12-06      |         355 |       21995.09 |          646.91 |            2.94 |
|          41 | Customer 41 | 2024-06-20  |           27 | 2024-06-23       | 2025-04-12      |         293 |       20795.78 |          770.21 |            5.56 |
+-------------+-------------+-------------+--------------+------------------+-----------------+-------------+----------------+-----------------+-----------------+
10 rows in set (0.49 sec)
```

6c. Simulated materialized view: summary table + refresh procedure + scheduled event

SQL

```
CREATE TABLE seller_performance_summary (
  seller_id        INT UNSIGNED NOT NULL PRIMARY KEY,
  seller_name      VARCHAR(120) NOT NULL,
  total_sales      DECIMAL(14,2) NOT NULL,
  units_sold       BIGINT NOT NULL,
  avg_rating       DECIMAL(4,2) NULL,
  review_count     INT NOT NULL,
  return_rate_pct  DECIMAL(5,2) NOT NULL,
  refreshed_at     DATETIME NOT NULL,
  KEY idx_summary_sales (total_sales)
) ENGINE=InnoDB
```

OUTPUT

```
Query OK, 0 rows affected (0.01 sec)
```

SQL

```
CREATE PROCEDURE sp_refresh_seller_performance()
BEGIN
  -- Build off to the side, then swap with one atomic RENAME so readers never
  -- see an empty or half-filled table (TRUNCATE + INSERT would expose that gap).
  DROP TABLE IF EXISTS seller_performance_new;
  DROP TABLE IF EXISTS seller_performance_old;
  CREATE TABLE seller_performance_new LIKE seller_performance_summary;

  INSERT INTO seller_performance_new
    (seller_id, seller_name, total_sales, units_sold, avg_rating, review_count, return_rate_pct, refreshed_at)
  SELECT seller_id, seller_name, total_sales, units_sold, avg_rating, review_count, return_rate_pct, NOW()
  FROM v_seller_performance;

  RENAME TABLE seller_performance_summary TO seller_performance_old,
               seller_performance_new     TO seller_performance_summary;
  DROP TABLE seller_performance_old;
END
```

OUTPUT

```
Query OK, 0 rows affected (0.00 sec)
```

SQL

```
CALL sp_refresh_seller_performance()
```

OUTPUT

```
Query OK, 0 rows affected (0.41 sec)
```

Needs the scheduler on (SET GLOBAL requires privileges; also set in my.cnf to persist)

SQL

```
SET GLOBAL event_scheduler = ON
```

OUTPUT

```
Query OK, 0 rows affected (0.00 sec)
```

SQL

```
CREATE EVENT IF NOT EXISTS ev_refresh_seller_performance
  ON SCHEDULE EVERY 1 HOUR STARTS CURRENT_TIMESTAMP
  DO CALL sp_refresh_seller_performance()
```

OUTPUT

```
Query OK, 0 rows affected (0.00 sec)
```

Compare: the view recomputes from 100k lines every call; the summary just reads ~50 stored rows.

SQL

```
ANALYZE TABLE seller_performance_summary
```

OUTPUT

```
+---------------------------------+---------+----------+----------+
| Table                           | Op      | Msg_type | Msg_text |
+---------------------------------+---------+----------+----------+
| shop.seller_performance_summary | analyze | status   | OK       |
+---------------------------------+---------+----------+----------+
1 row in set (0.00 sec)
```

SQL

```
EXPLAIN SELECT * FROM v_seller_performance        ORDER BY total_sales DESC LIMIT 10
```

OUTPUT

```
+----+-------------+------------+------------+--------+-----------------------------------------------+--------------------+---------+--------------------+------+----------+---------------------------------+
| id | select_type | table      | partitions | type   | possible_keys                                 | key                | key_len | ref                | rows | filtered | Extra                           |
+----+-------------+------------+------------+--------+-----------------------------------------------+--------------------+---------+--------------------+------+----------+---------------------------------+
|  1 | PRIMARY     | s          | NULL       | ALL    | NULL                                          | NULL               | NULL    | NULL               |   50 |    100.0 | Using temporary; Using filesort |
|  1 | PRIMARY     | <derived4> | NULL       | ref    | <auto_key0>                                   | <auto_key0>        | 4       | shop.s.seller_id   |  788 |    100.0 | NULL                            |
|  1 | PRIMARY     | <derived3> | NULL       | ref    | <auto_key0>                                   | <auto_key0>        | 4       | shop.s.seller_id   |  102 |    100.0 | NULL                            |
|  3 | DERIVED     | p          | NULL       | index  | PRIMARY,fk_products_seller                    | fk_products_seller | 4       | NULL               | 2000 |    100.0 | Using index                     |
|  3 | DERIVED     | r          | NULL       | ref    | uq_review_once                                | uq_review_once     | 4       | shop.p.product_id  |    5 |    100.0 | NULL                            |
|  4 | DERIVED     | p          | NULL       | index  | PRIMARY                                       | fk_products_seller | 4       | NULL               | 2000 |    100.0 | Using index; Using temporary    |
|  4 | DERIVED     | oi         | NULL       | ref    | fk_items_order,fk_items_product,fk_items_seller | fk_items_product   | 4       | shop.p.product_id  |   49 |    100.0 | NULL                            |
|  4 | DERIVED     | o          | NULL       | eq_ref | PRIMARY                                       | PRIMARY            | 8       | shop.oi.order_id   |    1 |     80.0 | Using where                     |
+----+-------------+------------+------------+--------+-----------------------------------------------+--------------------+---------+--------------------+------+----------+---------------------------------+
8 rows in set (0.00 sec)
```

SQL

```
EXPLAIN SELECT * FROM seller_performance_summary  ORDER BY total_sales DESC LIMIT 10
```

OUTPUT

```
+----+-------------+---------------------------+------------+------+---------------+------+---------+------+------+----------+----------------+
| id | select_type | table                     | partitions | type | possible_keys | key  | key_len | ref  | rows | filtered | Extra          |
+----+-------------+---------------------------+------------+------+---------------+------+---------+------+------+----------+----------------+
|  1 | SIMPLE      | seller_performance_summary | NULL       | ALL  | NULL          | NULL | NULL    | NULL |   50 |    100.0 | Using filesort |
+----+-------------+---------------------------+------------+------+---------------+------+---------+------+------+----------+----------------+
1 row in set (0.00 sec)

/* VIEW vs SUMMARY TABLE
   +---------------------+-------------------------------------+-------------------------------------+
   |                     | View                                | Summary table (simulated mat. view)   |
   +---------------------+-------------------------------------+-------------------------------------+
   | Freshness           | Always current                      | Stale until next refresh              |
   | Read cost           | Re-runs joins/aggregates every query | Cheap point/range read, indexable     |
   | Write cost          | None                                | Refresh job load (full or incremental)|
   | Storage             | None                                | Duplicates aggregated data            |
   | Indexing            | Only via underlying tables          | Index exactly what dashboards filter  |
   | Complexity          | Trivial, no maintenance             | Needs refresh logic, monitoring, swap |
   +---------------------+-------------------------------------+-------------------------------------+
   Use a view for small data, ad-hoc analysis, or when correctness-now matters.
   Use a summary table for heavy aggregations read often where "as of an hour ago" is acceptable.
   MySQL has no native materialized views. For big tables, refresh incrementally (only recent
   months) instead of rebuilding everything. */
```

View vs summary table: trade-off

	View	Summary table (simulated materialized view)
Freshness	Always current	Stale until the next refresh
Read cost	Re-runs joins and aggregates every query (~289 ms here)	Reads ~50 stored rows (~0.31 ms here)
Write / maintenance cost	None	Refresh job, monitoring, swap logic
Storage	None	Duplicates aggregated data
Indexing	Only through the underlying tables	Can index exactly what dashboards filter on
Best for	Small data, ad-hoc analysis, real-time correctness	Heavy aggregations read often where 'as of the last hour' is acceptable

7. Window functions

RANK() partitioned by category lists the top 3 products per category; LAG() gives month-over-month growth; AVG() over a 3-row frame gives the 3-month moving average; MIN() OVER (PARTITION BY customer_id) assigns each customer's first-purchase cohort; NTILE(4) splits customers into spending quartiles. The top spending quartile (25% of customers) generates **41.2%** of net revenue.

7a. Top 3 products per category by revenue (RANK)

SQL

```
WITH product_revenue AS (
  SELECT l.category_id, l.product_id, SUM(l.net_revenue) AS revenue
  FROM v_sales_lines l
  GROUP BY l.category_id, l.product_id
), ranked AS (
  SELECT category_id, product_id, revenue,
         RANK() OVER (PARTITION BY category_id ORDER BY revenue DESC) AS rnk
  FROM product_revenue
)
SELECT c.name AS category, p.title, ROUND(r.revenue, 2) AS revenue, r.rnk
FROM ranked r
JOIN products   p ON p.product_id  = r.product_id
JOIN categories c ON c.category_id = r.category_id
WHERE r.rnk <= 3
ORDER BY c.name, r.rnk
```

OUTPUT

```
+----------------+-------------------------+-----------+-----+
| category       | title                   | revenue   | rnk |
+----------------+-------------------------+-----------+-----+
| Beauty         | Stainless Speaker 20    |  96699.76 |   1 |
| Beauty         | Wireless Bottle 156     |  81038.10 |   2 |
| Beauty         | Stainless Speaker 236   |  66307.95 |   3 |
| Books          | Premium Backpack 35     | 114349.58 |   1 |
| Books          | Portable Headphones 3   |  68640.00 |   2 |
| Books          | Ergonomic Keyboard 187  |  63151.40 |   3 |
| Electronics    | Stainless Backpack 104  |  63600.96 |   1 |
| Electronics    | Stainless Speaker 128   |  58712.00 |   2 |
| Electronics    | Wireless Lamp 96        |  54503.16 |   3 |
| Fashion        | Wireless Keyboard 6     |  92632.32 |   1 |
| Fashion        | Wireless Keyboard 78    |  66016.99 |   2 |
| Fashion        | Organic Backpack 70     |  58422.06 |   3 |
| Garden         | Ergonomic Keyboard 7    | 117317.02 |   1 |
| Garden         | Ergonomic Keyboard 79   |  93852.00 |   2 |
... (10 more rows - full output in outputs/ folder of the repo)
+----------------+-------------------------+-----------+-----+
24 rows in set (0.56 sec)
```

RANK() gives ties the same rank, so a category can return >3 rows. Use ROW_NUMBER() for exactly 3, or DENSE_RANK() to avoid gaps after ties.

7b. Month-over-month growth (LAG)

SQL

```
WITH monthly AS (
  SELECT order_month, SUM(net_revenue) AS revenue
  FROM v_sales_lines GROUP BY order_month
)
SELECT order_month,
       ROUND(revenue, 2) AS revenue,
       ROUND(LAG(revenue) OVER (ORDER BY order_month), 2) AS prev_month_revenue,
       ROUND(100 * (revenue - LAG(revenue) OVER (ORDER BY order_month))
             / NULLIF(LAG(revenue) OVER (ORDER BY order_month), 0), 1) AS mom_growth_pct
FROM monthly
ORDER BY order_month
```

OUTPUT

```
+-------------+------------+--------------------+----------------+
| order_month | revenue    | prev_month_revenue | mom_growth_pct |
+-------------+------------+--------------------+----------------+
| 2023-01-01  |  170717.57 |               NULL |           NULL |
| 2023-02-01  |  376445.68 |          170717.57 |          120.5 |
| 2023-03-01  |  642341.72 |          376445.68 |           70.6 |
| 2023-04-01  |  742659.61 |          642341.72 |           15.6 |
| 2023-05-01  |  881485.16 |          742659.61 |           18.7 |
| 2023-06-01  |  941819.45 |          881485.16 |            6.8 |
| 2023-07-01  | 1105736.59 |          941819.45 |           17.4 |
| 2023-08-01  | 1106469.77 |         1105736.59 |            0.1 |
| 2023-09-01  | 1112678.05 |         1106469.77 |            0.6 |
| 2023-10-01  | 1138624.62 |         1112678.05 |            2.3 |
| 2023-11-01  | 1209326.59 |         1138624.62 |            6.2 |
| 2023-12-01  | 1265656.76 |         1209326.59 |            4.7 |
| 2024-01-01  | 1254797.08 |         1265656.76 |           -0.9 |
| 2024-02-01  | 1169555.78 |         1254797.08 |           -6.8 |
... (29 more rows - full output in outputs/ folder of the repo)
+-------------+------------+--------------------+----------------+
43 rows in set (0.34 sec)
```

7c. 3-month moving average of revenue ROWS counts physical rows, so it is only correct when no month is missing. Our data has every month; for gappy data use RANGE BETWEEN INTERVAL 2 MONTH PRECEDING AND CURRENT ROW.

SQL

```
WITH monthly AS (
  SELECT order_month, SUM(net_revenue) AS revenue
  FROM v_sales_lines GROUP BY order_month
)
SELECT order_month,
       ROUND(revenue, 2) AS revenue,
       ROUND(AVG(revenue) OVER w, 2) AS moving_avg_3m,
       COUNT(*)           OVER w     AS months_in_window      -- 1,2 = warm-up; 3 = full window
FROM monthly
WINDOW w AS (ORDER BY order_month ROWS BETWEEN 2 PRECEDING AND CURRENT ROW)
ORDER BY order_month
```

OUTPUT

```
+-------------+------------+--------------+------------------+
| order_month | revenue    | moving_avg_3m | months_in_window |
+-------------+------------+--------------+------------------+
| 2023-01-01  |  170717.57 |     170717.57 |                1 |
| 2023-02-01  |  376445.68 |     273581.63 |                2 |
| 2023-03-01  |  642341.72 |     396501.66 |                3 |
| 2023-04-01  |  742659.61 |     587149.00 |                3 |
| 2023-05-01  |  881485.16 |     755495.50 |                3 |
| 2023-06-01  |  941819.45 |     855321.41 |                3 |
| 2023-07-01  | 1105736.59 |     976347.07 |                3 |
| 2023-08-01  | 1106469.77 |    1051341.94 |                3 |
| 2023-09-01  | 1112678.05 |    1108294.80 |                3 |
| 2023-10-01  | 1138624.62 |    1119257.48 |                3 |
| 2023-11-01  | 1209326.59 |    1153543.09 |                3 |
| 2023-12-01  | 1265656.76 |    1204535.99 |                3 |
| 2024-01-01  | 1254797.08 |    1243260.14 |                3 |
| 2024-02-01  | 1169555.78 |    1230003.21 |                3 |
... (29 more rows - full output in outputs/ folder of the repo)
+-------------+------------+--------------+------------------+
43 rows in set (0.26 sec)
```

7d. First-purchase cohort per customer (MIN OVER PARTITION BY)

SQL

```
SELECT DISTINCT customer_id,
       MIN(order_date) OVER (PARTITION BY customer_id)                    AS first_purchase_date,
       DATE_FORMAT(MIN(order_date) OVER (PARTITION BY customer_id), '%Y-%m') AS cohort
FROM orders
WHERE status <> 'cancelled'
ORDER BY customer_id
LIMIT 20
```

OUTPUT

```
+-------------+---------------------+---------+
| customer_id | first_purchase_date | cohort  |
+-------------+---------------------+---------+
|           1 | 2023-06-22          | 2023-06 |
|           2 | 2023-02-12          | 2023-02 |
|           3 | 2023-07-06          | 2023-07 |
|           4 | 2023-10-05          | 2023-10 |
|           5 | 2023-07-03          | 2023-07 |
|           6 | 2023-10-29          | 2023-10 |
|           7 | 2023-03-14          | 2023-03 |
|           8 | 2023-01-22          | 2023-01 |
|           9 | 2024-02-08          | 2024-02 |
|          10 | 2023-05-29          | 2023-05 |
|          11 | 2024-07-04          | 2024-07 |
|          12 | 2024-07-14          | 2024-07 |
|          13 | 2024-04-10          | 2024-04 |
|          14 | 2023-12-17          | 2023-12 |
... (6 more rows - full output in outputs/ folder of the repo)
+-------------+---------------------+---------+
20 rows in set (0.13 sec)
```

7e. Spending quartiles (NTILE) - quartile 1 = top spenders

SQL

```
WITH spend AS (
  SELECT customer_id, SUM(net_revenue) AS total_spend
  FROM v_sales_lines GROUP BY customer_id
), q AS (
  SELECT customer_id, total_spend,
         NTILE(4) OVER (ORDER BY total_spend DESC) AS spend_quartile
  FROM spend
)
SELECT spend_quartile,
       COUNT(*)                        AS customers,
       ROUND(MIN(total_spend), 2)      AS min_spend,
       ROUND(MAX(total_spend), 2)      AS max_spend,
       ROUND(AVG(total_spend), 2)      AS avg_spend,
       ROUND(100 * SUM(total_spend) / SUM(SUM(total_spend)) OVER (), 1) AS pct_of_revenue
FROM q
GROUP BY spend_quartile
ORDER BY spend_quartile
```

OUTPUT

```
+----------------+-----------+-----------+-----------+-----------+----------------+
| spend_quartile | customers | min_spend | max_spend | avg_spend | pct_of_revenue |
+----------------+-----------+-----------+-----------+-----------+----------------+
|              1 |      1250 |   7530.03 |  31058.47 |   9673.30 |           41.2 |
|              2 |      1250 |   5499.04 |   7527.57 |   6438.43 |           27.4 |
|              3 |      1249 |   3833.63 |   5495.07 |   4670.40 |           19.9 |
|              4 |      1249 |     16.30 |   3833.33 |   2710.13 |           11.5 |
+----------------+-----------+-----------+-----------+-----------+----------------+
4 rows in set (0.36 sec)
```

Conclusion

The project delivers a normalised marketplace schema and a set of analytics that answer sales, satisfaction and customer-behaviour questions. Index choices were validated with EXPLAIN and EXPLAIN ANALYZE: composite indexes removed full table scans and filesorts, FULLTEXT replaced wildcard scans, and an index on the low-selectivity status column was shown to be a poor investment. A view keeps results always current, while a refreshed summary table trades freshness for much faster reads. Window functions produced rankings, growth, moving averages, cohorts and quartiles without self-joins.

Limitations: the data is synthetic, benchmarks come from a small dataset on one machine, and the summary table refreshes the whole result rather than incrementally.